

HANDOFF — Asystent AI eMMa (emmastudio.pl)

Dokument przekazania do nowej sesji w Claude Code (Windows).

Wklej ten plik jako pierwszy kontekst. Zawiera wszystko, co ustaliliśmy — nie trzeba wracać do poprzedniej rozmowy.

Data: 29.08.2026 · Autor projektu: Artur Krawczyk
(Arte Creative Studio)

1. Czym jest projekt

Widget czatu AI dla strony szkoły językowej **eMMa** — **Prywatne Studio Języków Obcych** (Poznań, emmastudio.pl).

Boczny przycisk przy krawędzi ekranu → rozwija okno rozmowy w brandingu strony → animowana maskotka (pluszowy triceratops) reaguje mimiką na treść rozmowy → historia zapisuje się w `localStorage` przeglądarki użytkownika.

Rola asystenta: doradca edukacyjny, nie sprzedawca. Odpowiada na pytania o ofertę i cennik, prowadzi do decyzji o lekcji próbnej, udziela mini-lekcji językowych jako próbki jakości nauczania.

2. Decyzje techniczne (podjęte, nie do przedyskutowania od nowa)

Obszar	Decyzja	Dlaczego
Model	Google Gemini gemini-2.5- flash , darmowy tier AI Studio	wymóg klienta: zero kosztów stałych
Wywołanie API	serverless proxy (Vercel Edge Function), klucz w env	klucz w kodzie klienckim = wyciek i wyczyszczony limit w tydzień
Frontend	vanilla JS + CSS, dwa pliki, bez zależności	strona ma nie stracić na Core Web Vitals

Obszar	Decyzja	Dlaczego
Pamięć	<code>localStorage</code> , klucz <code>emma_chat_v1</code>	wymóg klienta, brak backendu
Awatar	sprite sheet PNG + <code>background- position</code>	Artur dostarczył 4 arkusze, ~36 klatek maskotki
Baza wiedzy	statyczny <code>knowledge.json</code> generowany ze scrapowania sitemapy	model nie zna oferty, musi dostać ją w prompcie
Kolor brandowy	<code>#133B47</code> (głęboka morska zieleń)	<code>meta-theme- color</code> ze strony

Krytyczne ograniczenia

1. Klucz API nigdy w przeglądarce. Widget woła `/api/emma` , dopiero proxy woła Gemini.
2. Proxy musi mieć: rate limit po IP (20 wiad. / 10 min), limit długości wiadomości (600 znaków), limit historii (ostatnie 12 tur), allowlist `origin: https://emmastudio.pl` .

3. Darmowy tier ma dzienny cap. Przy 429 widget pokazuje formularz kontaktowy, nigdy surowy błąd.
 4. Asystent nie może wymyślać cen, terminów, nazwisk lektorów. Brak danych → prośba o kontakt z sekretariatem.
 5. RODO: informacja o zapisie rozmowy lokalnie + widoczny przycisk „Wyczyść rozmowę”.
-

3. Co wiemy o stronie

Potwierdzone z metatagów emmastudio.pl:

- eMMa — Prywatne Studio Języków Obcych, Poznań. Podmiot: Prywatne Studio Języków Obcych EMMA Faustyna Krawczyk
- Działa od 1992 roku
- Języki: angielski i hiszpański
- Grupy kameralne 4–8 osób
- Dzieci, młodzież, dorośli, firmy; kursy grupowe i indywidualne
- Egzaminy: FCE, CAE, IELTS, TOEFL; angielski biznesowy
- `theme-color: #133B47`

Ze starej wersji strony (emmastudio.com,
WordPress — WYMAGA WERYFIKACJI, może być

nieaktualne):

- kursy typowe: 60 minut, 2× w tygodniu (pon./śr. lub wt./czw.)
- maksymalna liczebność grup: 8 osób
- podział grup wg wieku i poziomu — test i rozmowa kwalifikacyjna
- świadectwa ukończenia kursu

Problem do rozwiązania: emmastudio.pl renderuje treść po stronie klienta. Zwykły `fetch` zwraca sam szkielet HTML z metatagami. Dlatego scraper używa Playwrighta, nie `fetch`.

Nieznane, do uzupełnienia ręcznie: cennik, harmonogram, opisy poziomów, zasady lekcji próbnej, kadra, dane kontaktowe sekretariatu, regulamin (rezygnacja, odrabianie zajęć, płatności).

4. Co już powstało

Pliki do wgrania do nowego projektu:

Plik	Status	Zawartość
emma-asystent-ai-prompt.md	gotowe	Część A: brief wykonawczy (architektura, mapowanie

Plik	Status	Zawartość
		awatara, schemat localStorage, UI). Część B: pełny system prompt asystenta — 12 tagów emocji, protokół profilowania leada, reguły bezpieczeństwa, 5 przykładów few-shot
scrape- emma.mjs	gotowe, przetestowane	Scraper sitemapy na Playwrightcie. Obsługuje sitemap index, filtruje zasoby, statystycznie wykrywa boilerplate, flaguje ceny i kontakty, dzieli na fragmenty ≤1800 znaków

Plik	Status	Zawartość
scrape-emma- README.md	gotowe	instrukcja uruchomienia scrapera
emma- knowledge- scraper.ipynb	do wyrzucenia	notatnik Colab – obejście na czas pracy z tabletu, na Windowsie zbędny

Scraper przetestowany na lokalnych fixture'ach:
parsowanie sitemapy, filtrowanie obcych domen i
zasobów, ekstrakcja sekcji, wykrywanie cen (1200
zł , 120 zł) i telefonu, usuwanie powtarzalnych
linii. Nie był uruchomiony przeciw prawdziwej
stronie.

5. Plan dalszych prac

Krok 1 – uruchomić scraper (pierwsze
zadanie w nowej sesji)

```
mkdir emma-knowledge && cd emma-knowledge
npm init -y && npm pkg set type=module
npm i playwright
```

```
npx playwright install chromium
# skopiuj scrape-emma.mjs do katalogu
node scrape-emma.mjs --max 3 --verbose      #
test
node scrape-emma.mjs --verbose              #
pełny przebieg
```

Wynik w `./data: knowledge.json`, `knowledge-block.txt`, `knowledge-review.md`.

Punkt kontrolny: przeczytać `knowledge-review.md`.

Jeśli sekcja „Strony bez treści” jest długa — podnieść `--timeout 60000`, zmniejszyć `--concurrency 1`. Jeśli nadal pusto — dodać czekanie na konkretny selektor treści w funkcji `scrapeWithBrowser`.

Krok 2 — uzupełnić bazę wiedzy

Wkleić do sekcji `WIEDZA 0 SZKOLE` w system prompcie treść z `knowledge-block.txt` + ręcznie dopisać to, czego nie ma na stronie (cennik, harmonogram, regulamin). Zweryfikować wszystkie ceny z Faustyną.

Krok 3 — proxy na Vercelu

`api/emma.js` — Edge Function. Rate limit, allowlist Origin, walidacja długości, obcięcie historii do 12 tur, mapowanie błędu 429 na przyjazny komunikat. Klucz w `GEMINI_API_KEY`.

Krok 4 – widget

`emma-widget.js` + `emma-widget.css`:

- side tab prawa krawędź, `#133B47`, napis „Zapytaj eMMę”
- okno 380×560 desktop, fullscreen <640px, `border-radius: 20px`
- dymki: user prawa (brand), asystent lewa (jasne tło)
- chipsy startowe: „Kurs dla dziecka”, „Angielski dla mnie”, „Szkolenie dla firmy”, „Cennik”, „Lekcja próbna”
- a11y: `role="dialog"`, `aria-live="polite"`, focus trap, Esc zamyka, kontrast $\geq 4.5:1$
- lazy-init dopiero po kliknięciu w tab (Core Web Vitals)

Krok 5 – silnik awatara

Parsowanie tagu `/^\s*\([([A-Z_]+)\]\s*/`, tag **zdejmowany z tekstu przed renderem**. Stan spoczynku `NEUTRAL` z subtelnym oddechem. Podczas oczekiwania `THINKING`. Nieznany tag → `NEUTRAL`. `prefers-reduced-motion` wyłącza animacje.

Do zrobienia: pociąć 4 arkusze sprite'ów, ustalić siatkę i wypełnić `AVATAR_MAP` (szkic w briefie, współrzędne do weryfikacji na realnych plikach).

Krok 6 — integracja i testy

Wpiąć `node scrape-emma.mjs` w build step. Test na mobile: klawiatura ekranowa nie może zasłaniać pola wpisywania. Test degradacji przy 429. Pomiar LCP/CLS przed i po.

6. Tagi emocji awatara (dla spójności w kodzie i prompcie)

```
SMILE GREETING THINKING EXCITED FUNNY  
NEUTRAL  
EMPATHY CURIOUS SURPRISED PROUD FOCUS  
SHY
```

Format odpowiedzi modelu — bezwzględnie:

```
[TAG_EMOCJI] Treść odpowiedzi.
```

7. Aktywa do przeniesienia

- 4 arkusze sprite'ów maskotki (PNG z przezroczystością, siatka 3×3, ~36 klatek łącznie)
- 1 obrazek bohaterski (maskotka z pucharem i konfetti, tło „GRATULACJE! SUKCES!”) — nadaje

się na stan `EXCITED` albo grafikę powitalną

- pliki wymienione w sekcji 4

8. Pierwsza wiadomość do nowej sesji

Wznawiam projekt asystenta AI dla emmastudio.pl. W załączeniu HANDOFF.md z pełnym kontekstem oraz pliki: system prompt i scraper. Zaczynamy od kroku 1 — uruchomienia scrapera przeciw prawdziwej stronie. Strona jest client-side rendered, więc spodziewaj się problemów z czasem ładowania treści.